U-Net treinada do zero para segmentação semântica urbana: um estudo de ablação no CamVid

Relatório de progresso — MAC5921/MAC0497 – Deep Learning (2026)
Docente: Nina S. T. Hirata

Elderclei Regis Reami

24 de outubro de 2026

1 Problema e perguntas de pesquisa

Segmentação semântica atribui uma classe a cada pixel da imagem. Estudamos a U-Net [1] treinada *sem pré-treino*, num dataset pequeno e desbalanceado, para isolar o que cada componente do projeto contribui quando não há representações prontas a herdar. Cada pergunta é respondida variando *um único fator* em relação a um baseline:

Q1 Arquitetura: quanto do desempenho vem das *skip connections*? Hipótese: sem elas o modelo perde detalhes espaciais, e a queda se concentra nas classes finas (poste, placa, ciclista).

Q2 Desbalanceamento: ponderar a cross-entropy por classe ou somar a perda Dice [4] melhora o mIoU? Hipótese: ganho nas classes raras, com possível perda leve nas dominantes.

Q3 Regularização: qual o efeito de data augmentation com apenas 367 imagens de treino? Hipótese: sem augmentation há mais overfitting (maior distância entre loss de treino e mIoU de validação).

Q4 Capacidade (opcional): como o mIoU varia com a largura da rede (1,9M, 7,8M e 31M parâmetros)?

2 Dataset e justificativa da escolha

O **CamVid** [2] contém frames de vídeo urbano anotados com 11 classes (mais *Void*, ignorada). Usamos a divisão do SegNet [3]: 367 imagens de treino, 101 de validação e 233 de teste, em 360×480. Motivos da escolha:

- **Viável do zero no orçamento:** é pequeno o bastante para cada treino levar minutos em uma GPU do Colab, o que permite várias variantes e seeds.
- **Multiclasse e desbalanceado** (Tab. 1): *Road* ocupa 31,7% dos pixels e *Bicyclist* 0,29% (razão ≈110×). Isso torna Q1 e Q2 informativas, ao contrário de datasets binários e fáceis (ex.: Carvana, onde as variantes saturam perto de Dice 0,99).
- **Divisão padronizada e amplamente usada**, o que permite comparar ordens de grandeza com a literatura.
- Domínio fora da saúde e sem cadastro para download (ao contrário do Cityscapes).

Tabela 1: Fração dos pixels de treino por classe (calculada nas 367 anotações).

Classe	%	Classe	%
Road	31,66	Car	5,87
Building	23,26	Pavement	4,49
Sky	16,85	SignSymbol	1,17
Tree	9,72	Fence	1,13
Void (ign.)	3,95	Pole	0,98
		Pedestrian	0,64
		Bicyclist	0,29

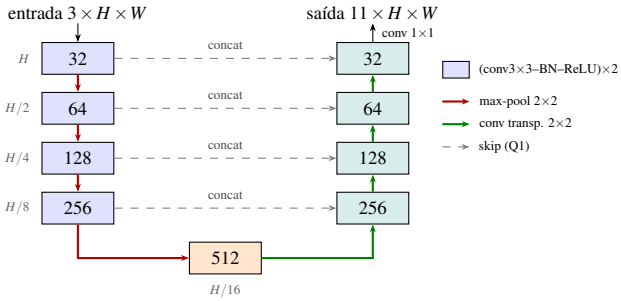


Figura 1: U-Net implementada (`src/model.py`). Os números indicam canais de saída de cada bloco; $H \times W$ é 256×256 no treino e 368×480 (com *padding*) na avaliação.

3 Metodologia

Modelo. U-Net com 4 níveis de *downsampling*, blocos conv3×3–BN–ReLU duplos, *upsampling* por convolução transposta e largura base de 32 filtros (32→512; 7,76M parâmetros), conforme a Fig. 1. Na variante sem skip, o decodificador recebe apenas o mapa reamostrado.

Diferenças em relação ao artigo original [1]. A topologia é a mesma (4 níveis, blocos de duas conv. 3×3, *max-pooling*, convolução transposta 2×2 que reduz os canais pela metade, concatenação e conv. 1×1 final). As diferenças são: (i) convoluções com *padding*, de modo que a saída tem o tamanho da entrada e a concatenação dispensa recortes, em vez de convoluções sem *padding* e da estratégia *overlap-tile*; (ii) metade da largura (32→512 em vez de 64→1024; 7,76M contra ≈31M parâmetros), adequada a 367 imagens; (iii) BatchNorm após cada convolução e sem dropout; (iv) treino com AdamW e batch 8, em vez de SGD com momento 0,99 e batch de uma imagem grande; (v) 11 classes, em vez de 2, e sem o mapa de pesos de borda do artigo.

Treino. AdamW (lr 10^{-3}, weight decay 10^{-4}), *cosine annealing*, 120 épocas, batch 8, precisão mista. Recortes aleatórios de 256×256 em todas as variantes; augmentation = flip horizontal + brilho/contraste. Pesos da CE ponderada por *median frequency balancing* [5].

Protocolo. Seleção do checkpoint pelo mIoU de validação (avaliado a cada 2 épocas); conjunto de teste avaliado *uma única vez*, na imagem inteira. **Métricas:** mIoU (principal), IoU por classe e acurácia de pixel, com média ± desvio-padrão sobre 3 seeds.

Ambiente computacional. Processamento feito diretamente no Google Colab, em uma máquina do Google Compute Engine com 12,7 GB de RAM, 112,6 GB de disco e GPU NVIDIA Tesla T4 de 15 GB; execução em Python 3.13.16.

4 Plano de experimentos

Planejamento de recursos. O custo total foi projetado antes de disparar o plano completo, a partir do tempo por época

Tabela 2: Variantes (cada uma muda um fator do baseline), 3 seeds cada.

Variante	Alteração	Pergunta
baseline	U-Net-32, CE, com aug.	—
no_skip	sem skip connections	Q1
wce	CE ponderada	Q2
ce_dice	CE + Dice	Q2
no_aug	sem augmentation	Q3
base16 / base64	largura 16 / 64	Q4

medido no smoke test (2 épocas):

$$T_{\text{total}} \approx N_{\text{var}} \times N_{\text{seeds}} \times E \times t_{\text{época}},$$

com $E = 120$ épocas. No smoke test, as 2 épocas (com validação em ambas) levaram 0,2 min, ou $t_{\text{época}} \approx 6$ s, o que projeta $120 \times 6\,\text{s} \approx 12$ min por treino. A projeção é conservadora, pois no plano a validação ocorre só a cada 2 épocas e a 1.ª época inclui o aquecimento da GPU. Na T4, os 15 treinos levaram **9,7 a 11,8 min** cada, dentro do previsto. As 5 variantes prioritárias × 3 seeds (15 treinos) somam, portanto, ≈**2,3–3,0 h** de GPU. Q4 roda se houver saldo (base64, com ≈4× mais parâmetros, não escala linearmente). Checkpoints no Drive permitem dividir o plano em várias sessões.

5 Código

Repositório: `https://github.com/ereami/unet-camvid`. Uma função `run(Config)` treina qualquer variante, com curvas em CSV e checkpoints retomáveis no Drive; o notebook `camvid_unet.ipynb` executa o roteiro completo no Colab.

6 Resultados (Q1–Q3)

As 5 variantes prioritárias foram concluídas com 3 seeds cada (15 treinos, 9,7 a 11,8 min cada na T4), em linha com a projeção da Seção 4.

Tabela 3: Teste, média sobre 3 seeds (± dp no mIoU). Colunas à direita: IoU por classe. Em negrito, o melhor resultado.

Variante	mIoU	Acc.	Build.	Pole	Ped.	Bicy.
baseline	0,532 ± 0,011	0,879	0,766	0,197	0,303	0,210
no_skip	0,471 ± 0,008	0,870	0,756	0,000	0,152	0,099
wce	0,492 ± 0,011	0,809	0,593	0,184	0,284	0,147
ce_dice	**0,581 ± 0,006**	**0,888**	**0,784**	**0,254**	**0,440**	**0,372**
no_aug	0,532 ± 0,010	0,876	0,751	0,200	0,302	0,216

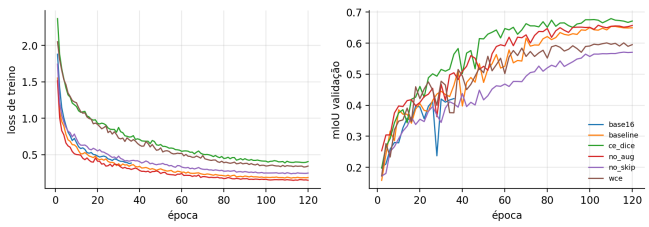


Figura 2: Loss de treino e mIoU de validação (seed 0). A curva `base16` corresponde a uma execução de Q4 interrompida na época 36.

Q1 (skip connections). Removê-las reduz o mIoU em 0,061 (≈6 dp). A perda se concentra nas classes finas: *Pole* vai a zero nas 3 seeds e *Pedestrian* e *Bicyclist* caem pela metade, enquanto *Road* e *Building* quase não mudam. A acurácia de pixel cai menos de 1 ponto, o que mostra por que ela engana sob desbalanceamento.

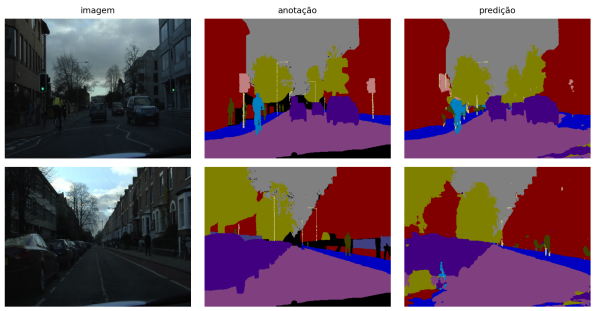


Figura 3: Exemplos do baseline no conjunto de teste (paleta oficial do CamVid).

Q2 (desbalanceamento). CE + Dice é a melhor variante (+0,049 de mIoU) e melhora *todas* as classes raras (*Bicyclist* 0,21→0,37). Já a CE ponderada *piora* o mIoU: os pesos altos das classes raras geram falsos positivos, que o IoU penaliza, e as classes dominantes perdem muito (*Building* 0,77→0,59; acurácia 0,88→0,81) sem ganho nas raras. Ponderar pela frequência, portanto, não é o mesmo que otimizar a sobreposição.

Q3 (augmentation). Sem flip e brilho/contraste o mIoU de teste é idêntico (0,532). A loss de treino fica mais baixa, mas o mIoU de validação não cai (Fig. 2), ou seja, não há overfitting mensurável. Ressalva: o recorte aleatório 256×256, mantido nas duas condições, já é uma augmentation forte; a hipótese de Q3 não se confirma *dado* esse recorte.

Validação vs. teste. O mIoU de teste fica ≈0,10–0,12 abaixo do de validação em todas as variantes, o que atribuímos ao viés de seleção do checkpoint e a diferenças entre as sequências de vídeo; como é sistemático, não altera a ordenação.

7 Próximos passos e riscos

- **Q4 (capacidade):** `base16` (3 seeds) e `base64`, que tem ≈4× mais parâmetros e custo estimado de 3–4× por treino.
- Separar o efeito do recorte aleatório em Q3 (treino em imagem inteira).
- Combinar CE + Dice com as demais conclusões numa configuração final.

8 Declaração de uso de IA

Neste trabalho foi utilizado o Claude Opus 5.5 (Anthropic) para identificar datasets adequados para segmentação, identificar perguntas de estudo pertinentes a esse tipo de problema e estruturar o código necessário para pré-processar o dataset e executar os experimentos que respondem a essas perguntas.

Referências

[1] O. Ronneberger, P. Fischer, T. Brox. U-Net: Convolutional networks for biomedical image segmentation. *MICCAI*, 2015.

[2] G. Brostow, J. Fauqueur, R. Cipolla. Semantic object classes in video: a high-definition ground truth database. *Pattern Recognition Letters*, 2009.

[3] V. Badrinarayanan, A. Kendall, R. Cipolla. SegNet: A deep convolutional encoder-decoder architecture for image segmentation. *IEEE TPAMI*, 2017.

[4] F. Milletari, N. Navab, S.-A. Ahmadi. V-Net: Fully convolutional neural networks for volumetric medical image segmentation. *3DV*, 2016.

[5] D. Eigen, R. Fergus. Predicting depth, surface normals and semantic labels with a common multi-scale convolutional architecture. *ICCV*, 2015.